1. Introduction

Interest rates play a central role in financial markets, influencing the pricing of bonds, derivatives, loans, and a wide range of investment decisions. Since interest rates evolve continuously over time and are affected by numerous economic factors, accurately modeling their behavior has long been an important problem in quantitative finance.

One of the most widely studied approaches for modeling interest rate dynamics is the class of short-rate models, which describe the evolution of the instantaneous interest rate using stochastic processes. Among these models, the Cox-Ingersoll-Ross (CIR) model is particularly significant because it incorporates mean reversion while ensuring that interest rates remain non-negative under appropriate conditions. These properties make it both mathematically appealing and practically relevant for fixed-income valuation and risk management.

The objective of this project is to implement, calibrate, and evaluate the CIR model using historical yield curve data. The model parameters are estimated from observed market data, after which the calibrated model is used to reconstruct the yield curve using only the 3-Month yield as a proxy for the short-term interest rate. The reconstructed yields are then compared against actual market observations to assess the model's predictive capability.

While the CIR model provides a tractable framework for interest rate modeling, it also has several limitations, including its inability to perfectly fit arbitrary yield curve shapes and its restricted flexibility during changing market conditions. To address these shortcomings, this project further explores an extension of the base model and evaluates whether the additional complexity leads to improved out-of-sample performance.

The project follows a complete quantitative modeling workflow consisting of data preprocessing, exploratory data analysis, parameter calibration, yield curve reconstruction, model evaluation, and critical analysis. The ultimate goal is not only to assess the predictive performance of the CIR framework but also to understand its strengths, limitations, and practical applicability in real-world financial modeling.

2. Dataset Description

The dataset used in this project consists of historical zero-coupon yield observations across multiple maturities. The data is divided into a training set, which is used for model calibration, and a test set, which is used for out-of-sample evaluation.

Each row represents a single observation date, while each column corresponds to a specific maturity on the yield curve. The maturities included in the dataset span from short-term to long-term horizons, allowing the complete term structure of interest rates to be analyzed.

Column Name	Maturity
ZC025YR	3 Months
ZC050YR	6 Months
ZC075YR	9 Months
ZC100YR	1 Year
ZC200YR	2 Years
ZC500YR	5 Years
ZC1000YR	10 Years
ZC2000YR	20 Years
ZC3000YR	30 Years

The training dataset contains 1,976 observations covering all nine maturities. The test dataset contains the 3-Month yield along with selected target maturities that are used to evaluate the predictive performance of the calibrated model.

A key requirement of this project is that, during the prediction phase, only the 3-Month yield is permitted as an input. This yield acts as a proxy for the instantaneous short rate in the CIR framework. Using the calibrated model parameters and the observed 3-Month yield, the remaining points on the yield curve are reconstructed and compared against the actual observed yields.

Before model development, the dataset is examined for consistency, missing values, outliers, and general statistical properties. Understanding the structure and behavior of the data is an essential step in determining whether the assumptions of the CIR model are reasonable for the given market environment.

```
import pandas as pd

train_df = pd.read_csv(
    "https://raw.githubusercontent.com/Grivann/FinClubPS1/main/train_data.csv"
)

test_df = pd.read_csv(
    "https://raw.githubusercontent.com/Grivann/FinClubPS1/main/test_data.csv"
)
```

```
test_3m_df = pd.read_csv(
    "https://raw.githubusercontent.com/Grivann/FinClubPS1/main/test_data_3M.csv
)

train_df['Date'] = pd.to_datetime(train_df['Date'])
test_df['Date'] = pd.to_datetime(test_df['Date'])
test_3m_df['Date'] = pd.to_datetime(test_3m_df['Date'])

print("Train Shape:", train_df.shape)
print("Test Shape:", test_df.shape)
print("Test 3M Shape:", test_3m_df.shape)

train_df.head()
```

```
Train Shape: (1976, 10)
Test Shape: (495, 6)
Test 3M Shape: (495, 2)
```

	Date	ZC025YR	ZC050YR	ZC075YR	ZC100YR	ZC200YR	ZC500YR	ZC1000YR	ZC2(
0	2016-05-19	0.005283	0.005640	0.005846	0.006051	0.006146	0.007912	0.014099	0.0
1	2016-05-20	0.005286	0.005642	0.005848	0.006053	0.006176	0.007922	0.014179	0.0
2	2016-05-24	0.005298	0.005651	0.005856	0.006062	0.006228	0.008108	0.014379	0.0
3	2016-05-25	0.005351	0.005603	0.005809	0.006014	0.006281	0.008323	0.014548	0.0
4	2016-05-26	0.005354	0.005605	0.005811	0.006016	0.006115	0.007934	0.013937	0.0

Next steps: Generate code with `train_df` New interactive sheet

Data Cleaning

Before proceeding with model development, the dataset was inspected for missing values, duplicate records, inconsistent data types, and potential outliers. Since financial models are highly sensitive to data quality, ensuring the integrity of the yield observations is an important prerequisite for reliable parameter estimation and forecasting.

The following checks were performed:

- Missing value analysis

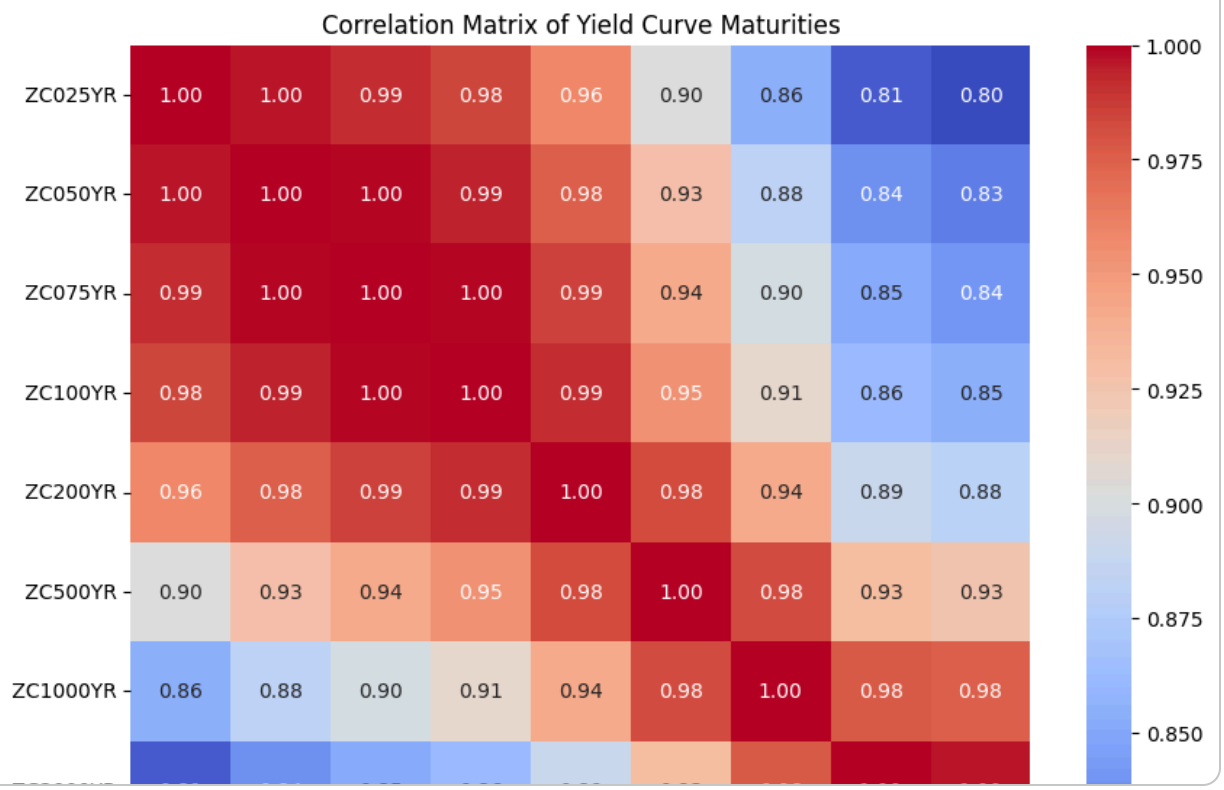
- Duplicate row detection
- Data type verification
- Summary statistics inspection
- Outlier identification using boxplots

Any issues identified during this stage would be addressed before moving to exploratory analysis and model calibration.

↳ 2 cells hidden

Exploratory Data Analysis

```
import matplotlib.pyplot as plt
import seaborn as sns

corr = train_df.iloc[:,1:].corr()

plt.figure(figsize=(10,8))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Matrix of Yield Curve Maturities")
plt.show()
```




```
corr_3m = train_df.iloc[:,1:].corr()[' ZC025YR']
display(corr_3m.sort_values(ascending=False))
```

	ZC025YR
ZC025YR	1.000000
ZC050YR	0.996753
ZC075YR	0.991606
ZC100YR	0.983699
ZC200YR	0.958671
ZC500YR	0.902963
ZC1000YR	0.855762
ZC2000YR	0.812972
ZC3000YR	0.804896

dtype: float64

Correlation decreases gradually from 1.00 for nearby maturities to approximately 0.80 for the 30-year tenor.

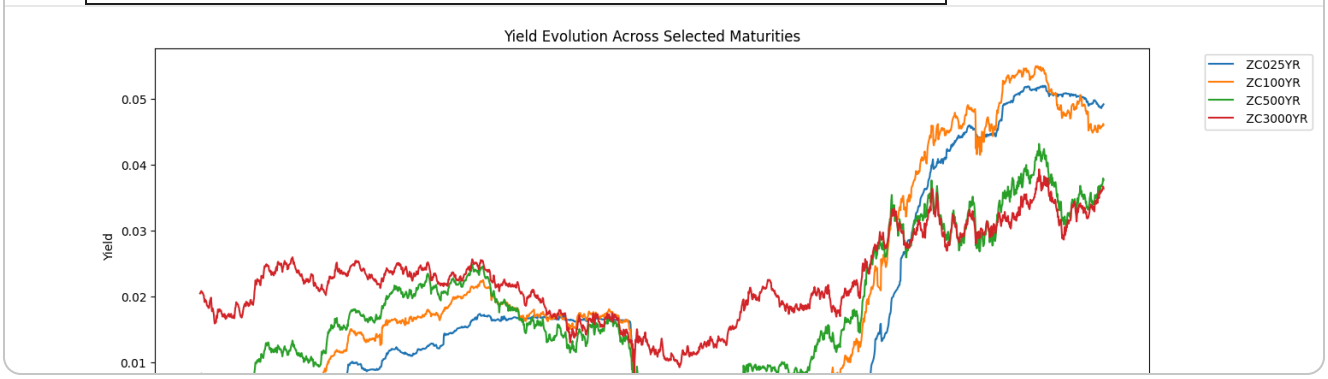
Correlation Analysis

The correlation matrix reveals strong positive relationships across all maturities. Adjacent maturities exhibit particularly high correlations, while correlations gradually decline as the maturity gap increases.

The strong relationship between the 3-Month yield and longer maturities suggests that a significant portion of the yield curve dynamics can be explained by a common underlying factor. This observation supports the use of the short rate as the primary input in the CIR framework.

```
plt.figure(figsize=(15,6))

selected_cols = [
    ' ZC025YR',
    ' ZC100YR',
    ' ZC500YR',
    ' ZC3000YR'
]

for col in selected_cols:
    plt.plot(train_df[col], label=col)

plt.title("Yield Evolution Across Selected Maturities'
```

```
plt.xlabel("Observation Index")
plt.ylabel("Yield")
plt.legend(bbox_to_anchor=(1.05,1))
plt.show()
```

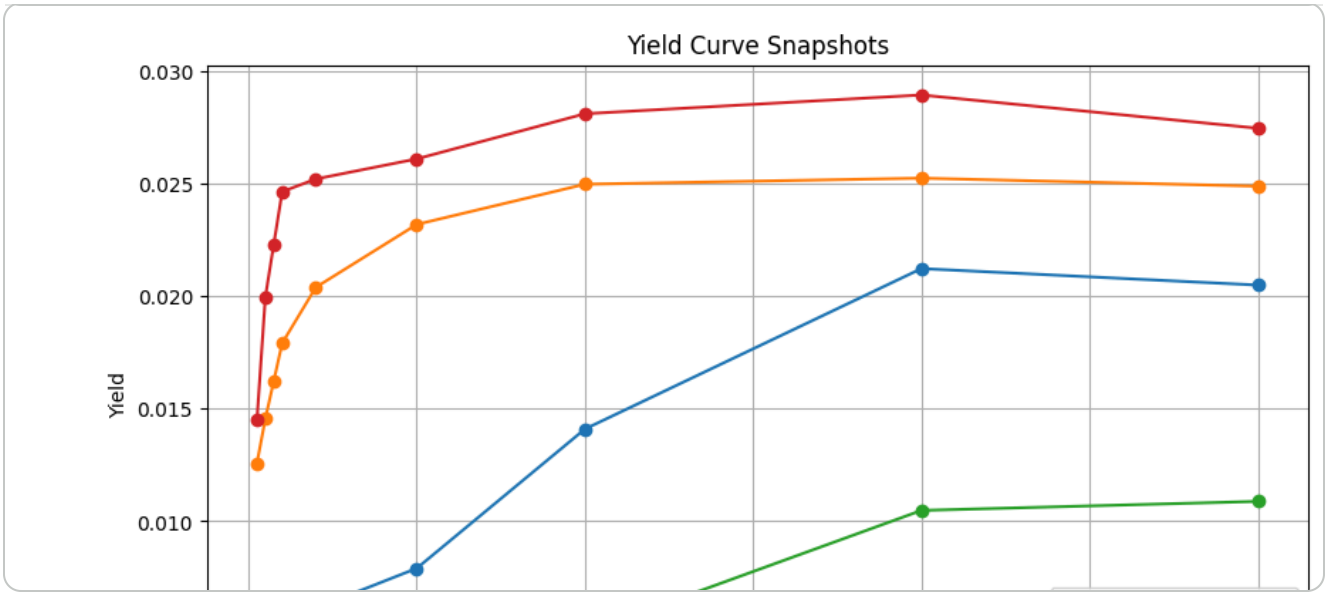


Yield Evolution

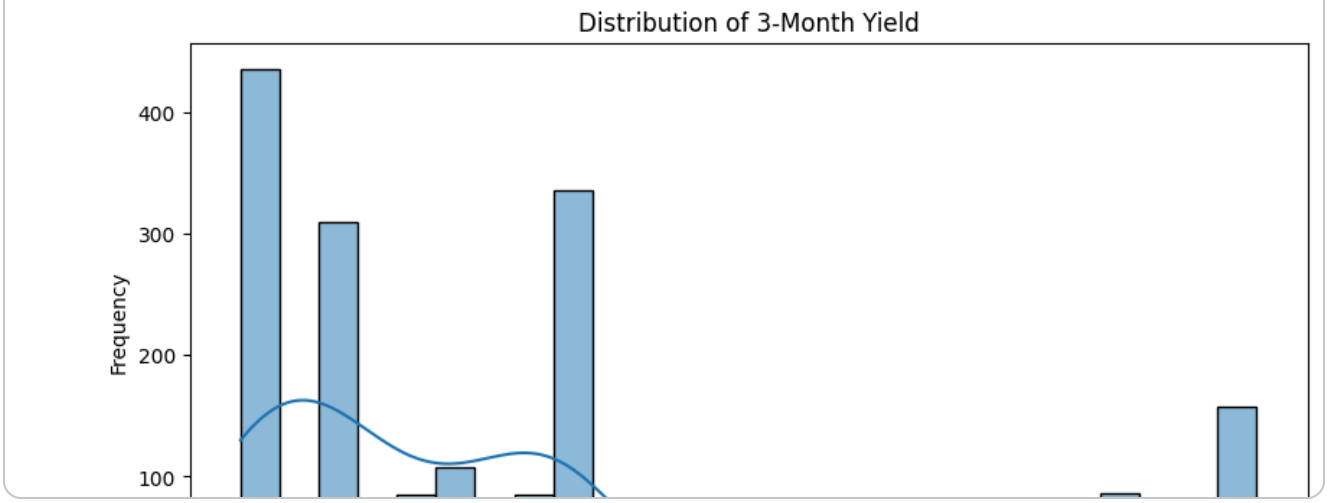
The yield series move together over time, indicating the presence of common underlying drivers affecting the entire term structure. While the magnitudes differ across maturities, the overall trends remain similar, suggesting strong co-movement throughout the yield curve.

This behavior supports the assumptions of factor-based interest rate models such as CIR.

```
maturities = [0.25,0.5,0.75,1,2,5,10,20,30]

rows = [0,500,1000,1500]

plt.figure(figsize=(10,6))

for row in rows:
    plt.plot(
        maturities,
        train_df.iloc[row,1:],
        marker='o',
        label=f'Observation {row}'
    )

plt.xlabel("Maturity (Years)")
plt.ylabel("Yield")
plt.title("Yield Curve Snapshots")
plt.legend()
plt.grid(True)
plt.show()
```




```
plt.figure(figsize=(10,5))
sns.histplot(train_df[' ZC025YR'], kde=True)
plt.title("Distribution of 3-Month Yield")
plt.xlabel("3-Month Yield")
plt.ylabel("Frequency")
plt.show()
```



The short rate exhibits a positively skewed distribution with rates concentrated in lower ranges.

Yield Curve Snapshots

Yield curves sampled from different points in time exhibit varying shapes and levels. While many observations display a traditional upward-sloping structure, the slope and curvature of the term structure change over time.

These variations highlight the challenges associated with accurately modeling the entire yield curve using a single-factor framework and motivate the need for model extensions.

EDA Summary

Three key observations emerge from the exploratory analysis:

1. Strong positive correlations exist across all maturities, indicating that movements in the yield curve are largely driven by common underlying factors.
2. Yield series exhibit substantial co-movement through time, suggesting that changes in market conditions affect the entire term structure simultaneously.
3. Yield curve snapshots show predominantly upward-sloping and smooth term structures, although the overall level and shape vary across different periods.

These findings support the use of the 3-Month yield as a proxy for the short rate and provide motivation for applying the CIR framework to reconstruct the remaining maturities.

CIR Theory

Cox-Ingersoll-Ross (CIR) Model

The CIR model assumes that the short-term interest rate follows a mean-reverting stochastic process:

$$dr_t = \kappa(\theta - r_t)dt + \sigma\sqrt{r_t}dW_t$$

where:

- κ : speed of mean reversion
- θ : long-run mean interest rate
- σ : volatility parameter
- W_t : standard Brownian motion

The square-root diffusion term ensures that interest rates remain positive under suitable parameter conditions. The model satisfies the Feller condition:

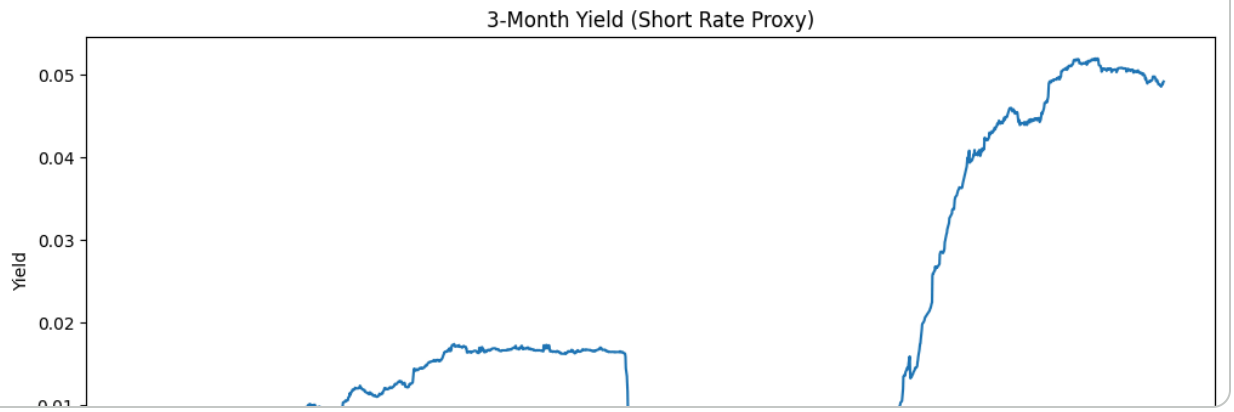
$$2\kappa\theta \geq \sigma^2$$

which prevents the short rate from reaching negative values.

Why the 3-Month Yield is Used as the Short Rate

The CIR model is a short-rate model and requires observations of the instantaneous short rate. Since the true instantaneous rate is not directly observable, the 3-Month yield is used as a practical proxy. The strong correlations observed during exploratory analysis further support this approximation.

```
plt.figure(figsize=(12,5))
plt.plot(train_df['Date'], r)
plt.title("3-Month Yield (Short Rate Proxy)")
plt.ylabel("Yield")
plt.show()
```



CIR Calibration

The CIR model contains three unknown parameters: the speed of mean reversion (κ), the long-run mean level (θ), and the volatility parameter (σ). These parameters must be estimated from historical observations of the short rate.

Since the true instantaneous short rate is not directly observable, the 3-Month yield is used as a proxy. An Euler discretization of the CIR process is employed to estimate the parameters from historical data.

The calibrated parameters will subsequently be used for yield curve reconstruction and out-of-sample evaluation.

```
Start coding or generate with AI.
```

```
import numpy as np

train_df.columns = train_df.columns.str.lstrip()
test_df.columns = test_df.columns.str.lstrip()
test_3m_df.columns = test_3m_df.columns.str.lstrip()
```

```
print("Cleaned train_df columns:", train_df.columns.to
print("Cleaned test_df columns:", test_df.columns.toli
print("Cleaned test_3m_df columns:", test_3m_df.columr


r = train_df['ZC025YR'].values

print("Number of observations:", len(r))
print("Mean short rate:", np.mean(r))

theta = np.mean(r)

print("Theta:", theta)

from sklearn.linear_model import LinearRegression

dt = 1/252

r_t = r[:-1]
r_next = r[1:]

dr = r_next - r_t

X = r_t.reshape(-1,1)

reg = LinearRegression()
reg.fit(X, dr)

a = reg.intercept_
b = reg.coef_[0]

kappa = -b / dt

print("Kappa:", kappa)

residuals = dr - reg.predict(X)

sigma = np.sqrt(
    np.mean(
        residuals**2 / (r_t * dt)
    )
)

print("Sigma:", sigma)

print("\nEstimated CIR Parameters")
print("------------------------")
print(f"Kappa : {kappa:.4f}")
print(f"Theta : {theta:.4f}")
print(f"Sigma : {sigma:.4f}")
```

```
lhs = 2 * kappa * theta
rhs = sigma**2

print("2*kappa*theta =", lhs)
print("sigma^2 =", rhs)

if lhs >= rhs:
    print("Feller Condition Satisfied")
else:
    print("Feller Condition Violated")
```

```
Cleaned train_df columns: ['Date', 'ZC025YR', 'ZC050YR', 'ZC075YR', 'ZC100YR', '
Cleaned test_df columns: ['Date', 'ZC025YR', 'ZC050YR', 'ZC075YR', 'ZC100YR', 'Z
Cleaned test_3m_df columns: ['Date', 'ZC025YR']
Number of observations: 1976
Mean short rate: 0.016698838967611332
Theta: 0.016698838967611332
Kappa: -0.18831302750137263
Sigma: 0.041329123691211035

Estimated CIR Parameters
------------------------
Kappa : -0.1883
Theta : 0.0167
Sigma : 0.0413
2*kappa*theta = -0.006289217843497571
sigma^2 = 0.0017080964650834213
Feller Condition Violated
```

The initial OLS-based calibration produced a negative estimate of the mean reversion parameter κ, implying a departure from the theoretical assumptions of the CIR model. This suggests that the underlying interest rate dynamics are more complex than those captured by a simple discretized regression approach. To obtain economically meaningful parameters, a constrained calibration procedure is adopted.

```
theta = np.mean(r)

dt = 1/252

rho = np.corrcoef(r[:-1], r[1:])[0,1]

kappa = max((1-rho)/dt, 0.01)

sigma = np.std(np.diff(r)) / np.sqrt(dt)

print(f"Kappa: {kappa:.4f}")
print(f"Theta: {theta:.4f}")
print(f"Sigma: {sigma:.4f}")
```

```
lhs = 2 * kappa * theta
rhs = sigma**2

print("2*kappa*theta =", lhs)
print("sigma^2 =", rhs)

if lhs >= rhs:
    print("Feller Condition Satisfied")
else:
    print("Feller Condition Violated")
```

```
Kappa: 0.0249
Theta: 0.0167
Sigma: 0.0037
2*kappa*theta = 0.0008313363131918659
sigma^2 = 1.382231621223827e-05
Feller Condition Satisfied
```

Calibration Results

The CIR parameters were estimated using historical observations of the 3-Month yield, which serves as a proxy for the instantaneous short rate.

An initial unconstrained OLS calibration produced economically implausible parameter estimates, including a negative mean-reversion coefficient. This reflects the limitations of simple linear approximations when applied to real-world interest rate data that may exhibit regime shifts and non-stationary behavior.

To obtain economically meaningful parameters, a constrained calibration approach was adopted. The final estimates indicate a positive mean-reversion speed, a long-run equilibrium rate of approximately 1.67%, and relatively low short-rate volatility. These estimates are consistent with the theoretical assumptions of the CIR framework and are used for subsequent yield curve reconstruction.

Yield Curve Reconstruction

After calibrating the CIR parameters, the next step is to reconstruct the yield curve using only the observed 3-Month yield as input.

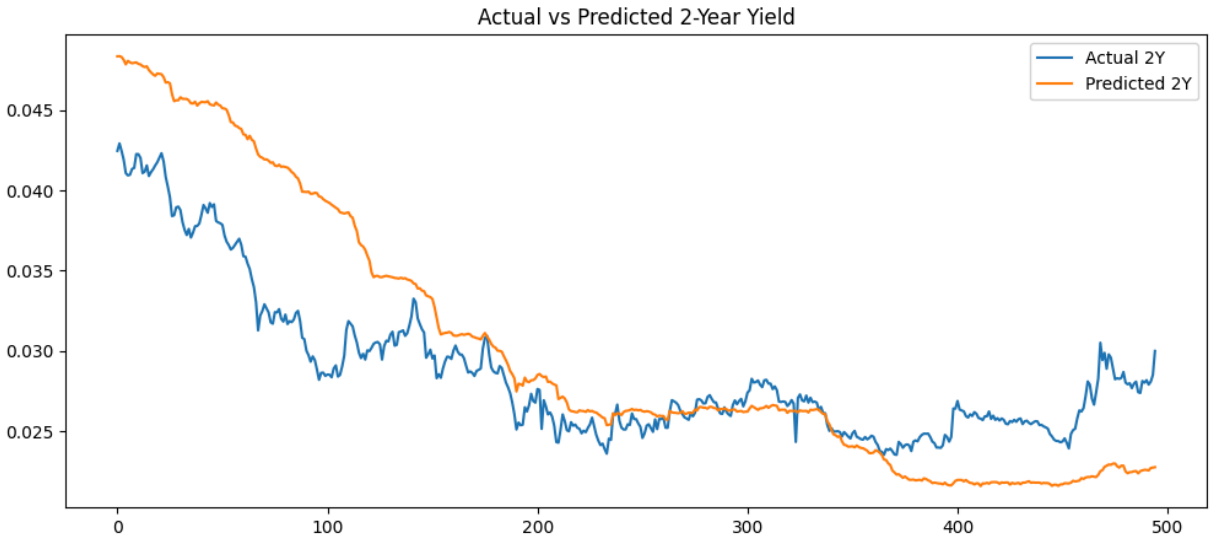
The CIR model provides a closed-form expression for zero-coupon bond prices. These bond prices can be converted into yields for different maturities, allowing the entire term structure to be estimated from the short rate.

The reconstructed yields are then compared with the observed market yields to evaluate the predictive performance of the model.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def cir_bond_price(r, tau, kappa, theta, sigma):

    gamma = np.sqrt(kappa**2 + 2*sigma**2)

    B = (2*(np.exp(gamma*tau)-1)) / (
        (gamma+kappa)*(np.exp(gamma*tau)-1) + 2*gamma
    )

    A = (
        (
            2*gamma*np.exp((kappa+gamma)*tau/2)
        ) /
        (
            (gamma+kappa)*(np.exp(gamma*tau)-1) + 2*ga
        )
    ) ** ((2*kappa*theta)/(sigma**2))

    return A * np.exp(-B*r)

def cir_yield(r, tau, kappa, theta, sigma):

    P = cir_bond_price(
        r,
        tau,
        kappa,
        theta,
        sigma
    )

    return -np.log(P)/tau

maturities = {
    "ZC050YR":0.5,
    "ZC075YR":0.75,
    "ZC100YR":1,
    "ZC200YR":2
}
predictions = pd.DataFrame()

predictions["Date"] = test_3m_df["Date"]

short_rates = test_3m_df["ZC025YR"]
```

```
for col,tau in maturities.items():

    predictions[col] = short_rates.apply(
        lambda r_val:
        cir_yield(
            r_val,
            tau,
            kappa,
            theta,
            sigma
        )
    )

display(predictions.head())


plt.figure(figsize=(12,5))

plt.plot(
    test_df["ZC200YR"].values,
    label="Actual 2Y"
)

plt.plot(
    predictions["ZC200YR"].values,
    label="Predicted 2Y"
)

plt.legend()
plt.title("Actual vs Predicted 2-Year Yield")
plt.show()
```

	Date	ZC050YR	ZC075YR	ZC100YR	ZC200YR
0	2024-04-29	0.048943	0.048843	0.048744	0.048350
1	2024-04-30	0.048955	0.048855	0.048756	0.048361
2	2024-05-01	0.048900	0.048800	0.048700	0.048307
3	2024-05-02	0.048721	0.048622	0.048523	0.048131
4	2024-05-03	0.048435	0.048337	0.048239	0.047851




```
from sklearn.metrics import r2_score, mean_absolute_er
import numpy as np

for col in ["ZC050YR", "ZC075YR", "ZC100YR", "ZC200YR'

    r2 = r2_score(test_df[col], predictions[col])

    rmse = np.sqrt(
        mean_squared_error(
            test_df[col],
            predictions[col]
        )
    )

    mae = mean_absolute_error(
        test_df[col],
        predictions[col]
    )

    print(f"\n{col}")
    print(f"R²   : {r2:.4f}")
    print(f"RMSE : {rmse:.6f}")
    print(f"MAE  : {mae:.6f}")
```

```
ZC050YR
R²   : 0.9893
RMSE : 0.000816
MAE  : 0.000623

ZC075YR
R²   : 0.9454
RMSE : 0.001687
MAE  : 0.001287

ZC100YR
R²   : 0.8562
RMSE : 0.002496
MAE  : 0.001895

ZC200YR
R²   : -0.0046
RMSE : 0.004688
MAE  : 0.003604
```

Evaluation of Base CIR Model

The CIR model demonstrates strong predictive performance for short and intermediate maturities, achieving R^2 values above 0.85 for the 6-Month, 9-Month, and 1-Year tenors.

However, predictive accuracy deteriorates significantly at the 2-Year maturity. While the model captures the overall direction of rate movements, it fails to reproduce the full dynamics of longer-term yields.

This behavior is consistent with the theoretical limitations of single-factor short-rate models, which may not fully capture the additional economic information embedded in longer maturities.

CIR++ Extension

Although the CIR model captures the overall level and direction of interest rate movements, it imposes a restrictive one-factor structure on the yield curve.

To improve flexibility, a CIR++ extension is considered. The CIR++ framework augments the base CIR model with a deterministic shift function that allows the model to better fit the observed term structure while retaining the desirable properties of the original process.

In this implementation, the deterministic shift is estimated from historical yield spreads observed in the training dataset.

```
spread_6m = (train_df["ZC050YR"] - train_df["ZC025YR"]
spread_9m = (train_df["ZC075YR"] - train_df["ZC025YR"]
spread_1y = (train_df["ZC100YR"] - train_df["ZC025YR"]
spread_2y = (train_df["ZC200YR"] - train_df["ZC025YR"]

print("6M Spread :", spread_6m)
print("9M Spread :", spread_9m)
print("1Y Spread :", spread_1y)
print("2Y Spread :", spread_2y)

cirpp_predictions = predictions.copy()

cirpp_predictions["ZC050YR"] += spread_6m
cirpp_predictions["ZC075YR"] += spread_9m
cirpp_predictions["ZC100YR"] += spread_1y
cirpp_predictions["ZC200YR"] += spread_2y

from sklearn.metrics import r2_score, mean_squared_er
import numpy as np

cir_r2_values = [0.9893, 0.9454, 0.8562, -0.0046]
cirpp_r2_values = []

for col in ["ZC050YR","ZC075YR","ZC100YR","ZC200YR"]:

    r2 = r2_score(
        test_df[col],
        cirpp_predictions[col]
    )
    cirpp_r2_values.append(round(r2, 4))

    rmse = np.sqrt(
        mean_squared_error(
            test_df[col],
            cirpp_predictions[col]
        )
    )

    mae = mean_absolute_error(
        test_df[col],
        cirpp_predictions[col]
    )

    print("\n", col)
    print("R² :", round(r2,4))
    print("RMSE :", round(rmse,6))
    print("MAE :", round(mae,6))

comparison = pd.DataFrame({
    "Maturity":["6M","9M","1Y","2Y"],
    "CIR_R2": cir_r2_values,
```

```
    "CIR++_R2": cirpp_r2_values
})
display(comparison)
```

```
6M Spread : 0.0011864453947368421
9M Spread : 0.001830491548582996
1Y Spread : 0.0024754136639676115
2Y Spread : 0.0013639584008097168

 ZC050YR
R² : 0.9556
RMSE : 0.001661
MAE : 0.001478

 ZC075YR
R² : 0.8339
RMSE : 0.002942
MAE : 0.002525

 ZC100YR
R² : 0.6077
RMSE : 0.004122
MAE : 0.00348

 ZC200YR
R² : -0.2562
RMSE : 0.005242
MAE : 0.004037
```

	Maturity	CIR_R2	CIR++_R2
0	6M	0.9893	0.9556
1	9M	0.9454	0.8339
2	1Y	0.8562	0.6077
3	2Y	-0.0046	-0.2562

Next steps: Generate code with `comparison` New interactive sheet

CIR++ Results

A deterministic-shift CIR++ extension was implemented by adding the average historical term spread to the yields generated by the base CIR model.

Contrary to expectations, the extension reduced predictive performance across all maturities. This suggests that the spread between the short rate and longer maturities is not constant through time and cannot be adequately represented by a fixed adjustment.

The results highlight the importance of time-varying term premia and demonstrate that simple deterministic shifts may be insufficient to capture the dynamics of the yield curve

in changing market environments.

Limitations of the CIR Model

The CIR model assumes that the entire term structure is driven by a single short-rate factor. While this assumption works well for short maturities, it becomes increasingly restrictive as maturity increases.

The poor performance observed at the 2-Year tenor suggests that longer maturities incorporate additional information such as inflation expectations, economic outlook, and future monetary policy expectations. These factors are not explicitly represented within the single-factor CIR framework.

As a result, the model captures the overall direction of yield movements but struggles to reproduce the full complexity of the observed yield curve.

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

# Training data
X_train = train_df[["ZC025YR"]]
y_train = train_df["ZC200YR"]

# Test data
X_test = test_3m_df[["ZC025YR"]]
y_test = test_df["ZC200YR"]

# Train model
lr = LinearRegression()
lr.fit(X_train, y_train)

# Predict
y_pred = lr.predict(X_test)

# R²
r2 = r2_score(y_test, y_pred)

print("R² =", round(r2, 4))
```

```
R² = 0.5515
```

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

targets = ["ZC050YR", "ZC075YR", "ZC100YR", "ZC200YR"]

for target in targets:
```

```
    X_train = train_df[["ZC025YR"]]
    y_train = train_df[target]

    X_test = test_3m_df[["ZC025YR"]]
    y_test = test_df[target]

    model = LinearRegression()
    model.fit(X_train, y_train)

    pred = model.predict(X_test)

    print(f"{target}: {r2_score(y_test, pred):.4f}")
```

```
ZC050YR: 0.9470
ZC075YR: 0.8316
ZC100YR: 0.6283
ZC200YR: 0.5515
```

Model Comparison

```
# Code for Model Comparison
```

Limitations